UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

FACULTY OF COMPUTING

SEMESTER 2/20252026

SECP3133 - HIGH PERFORMANCE DATA PROCESSING

SECTION 01

PROJECT 2: REAL TIME SENTIMENT ANALYSIS USING APACHE SPARK AND KAFKA

GROUP *Reporter* TEAM MEMBERS:

NAME	MATRIC ID
MUHAMMAD AFIQ DANIAL BIN ROZAIDIE	A23CS0117
AHMAD ADIB ZIKRI BIN A.MAZLAM	A23CS0205
DHESHIEGHAN A/L SARAVANA MOORTHY	A23CS0072

LECTURER:
Prof. Madya. Ts. Dr Mohd Shahizan bin Othman

SUBMITTED ON: 26 June 2026

Table of Contents

1.0 Introduction

1.1 Background

Social media platforms generate a large amount of unstructured public opinion every day, and YouTube comment sections are among the most useful sources of spontaneous audience feedback. For topics related to cost of living, inflation and lifestyle in Malaysia, viewers openly express their opinions, concerns and experiences, whether they feel positive, negative or neutral about living conditions in the country. Capturing and understanding this sentiment at scale is valuable because it helps show how people respond to current issues such as rising expenses, migration decisions, food prices and quality of life. However, analysing this data is technically challenging because YouTube comments are continuously produced in large volumes and are often informal, unstructured and written using mixed language. This project applies a high-performance data processing approach to solve that problem. Instead of analysing a small static sample, the system is designed to extract real YouTube comments from selected videos such as “Cost of living in Malaysia per month - Will you be moving to Malaysia?”, “Cost of Living in Malaysia 2026: The Real Numbers”, “Singapore vs Malaysia: Cost of Living 2026”, “INFLASI | Singapura Gusar, Malaysia Henti Eksport Ayam”, “Pros and Cons of Living in Kuala Lumpur” and “Pros and Cons of Living in Malaysia (2025)”. The extracted comments are cleaned, classified into positive, negative and neutral sentiment using a machine learning model, and processed through a distributed streaming pipeline built using Apache Kafka and Spark Structured Streaming. Additionally, Elasticsearch is used to store the classified results, while Kibana is used to visualize the sentiment findings. This makes the project locally relevant and suitable for analysing public sentiment toward Malaysia’s cost of living and lifestyle conditions.

1.2 Objectives

The objectives of this project are:

1. To collect a large representative dataset of real YouTube comments from selected videos related to cost of living, inflation and lifestyle in Malaysia using the YouTube Data API v3.

2. To preprocess and clean the raw comment text so that it is suitable for sentiment analysis and machine learning classification.
3. To assign sentiment labels which are positive, negative and neutral to the unlabelled YouTube comments.
4. To train and compare sentiment classification models and select the most efficient and accurate model for deployment.
5. To integrate the selected model into a real-time processing pipeline using Apache Kafka and Apache Spark Structured Streaming.
6. To store the classified sentiment results in Elasticsearch and present the findings through interactive Kibana dashboards.
7. To compare batch and streaming performance based on processing time, throughput, accuracy, memory usage and CPU usage.
8. To document the complete system architecture, implementation process and performance findings in a structured technical report format.

1.3 Project Scope

This project covers the complete end-to-end pipeline from raw YouTube comment collection to sentiment classification, real-time processing, storage and visualization. It includes data acquisition from selected YouTube videos, text preprocessing, sentiment labelling, machine learning model training and evaluation, distributed stream processing, Elasticsearch storage and Kibana dashboards. The scope is bounded as follows:

- **Data source:** The analysis is limited to text comments extracted from six selected YouTube videos related to cost of living, inflation and lifestyle in Malaysia. Comments from other platforms such as TikTok, Facebook, Instagram and X are not included.
- **Content type:** Only text-based YouTube comments are analysed. Video, audio, images, thumbnails and user profile details are out of scope because this project focuses only on text sentiment analysis.
- **Sentiment granularity:** Sentiment is treated as a three-class problem which are positive, neutral and negative. More detailed analysis such as emotion detection, sarcasm detection or aspect-based sentiment analysis is not covered.

- **Labelling approach:** The raw YouTube comments are prepared with sentiment labels so that they can be used for supervised machine learning. The labelled comments are then used to train and evaluate the sentiment classification model.
- **Modelling:** Sentiment classification models are trained and compared, and the strongest model is selected for deployment. The selected model uses text features from cleaned comments to classify sentiment accurately.
- **Processing pipeline:** The system implements both batch and streaming processing. Batch mode processes the full dataset at once, while streaming mode uses Apache Kafka and Apache Spark Structured Streaming to process comments in real time.
- **Storage and visualization:** The classified sentiment results are stored in Elasticsearch and presented through Kibana dashboards to show the sentiment distribution and overall findings.
- **Performance comparison:** Batch and streaming modes are compared based on processing time, throughput, accuracy, memory usage and CPU usage to evaluate the efficiency and real-time capability of the system.

2.0 Data Acquisition

2.1 Data Source

The primary data source for this project consists of YouTube comments collected from videos related to Malaysia. The dataset was scraped from six different YouTube videos and stored as a CSV file as "youtube_comments_all.csv". The comments cover a range of perspectives from both local Malaysians and international viewers discussing topics such as living in Malaysia, travel experiences, and social observations, making it a relevant and rich source for sentiment analysis in a Malaysian context.

The raw dataset contained 5,388 records across 8 columns which are video_id, user, comment, likes, published_at, reply_count, type, and parent_id. Comments were a mix of both top-level posts and replies, written in both English and Bahasa Malaysia language.

Table 2.1 shows the title of six YouTube videos that was scraped in this project as well as its description:

Table 2.1 Video References and Descriptions

No	Title	Description
1.	Cost of living in Malaysia per month - Will you be moving to Malaysia?	This video outlines the major monthly living expenses for a family of three relocating to Malaysia. It provides a comprehensive budget breakdown that goes beyond basic rent, highlighting realistic costs that expats often overlook, such as international schooling fees and travel expenses.
2.	Cost of Living in Malaysia 2026: The Real Numbers	Hosted by Nazrin Razali, this video provides an updated financial reality check for anyone looking to move to Malaysia. It details exact monthly numbers across different lifestyles (budget to luxury) for singles, couples, and families, covering everything from condos vs. landed housing to groceries, utilities, and hidden fees.
3.	Singapore vs Malaysia: Cost of	Featuring *MrMoneyTV*, this video breaks down

	Living 2026	the side-by-side living expenses between Singapore and Malaysia. It compares core components like food, rent, public transport, taxes, and healthcare to explore whether Singapore's stronger currency yields better net savings, or if the lifestyle tradeoffs make Malaysia a better choice.
4.	INFLASI | Singapura Gusar, Malaysia Henti Eksport Ayam	A news broadcast by *Buletin TV3* detailing the economic friction caused when Malaysia temporarily banned chicken exports to combat domestic inflation. The report focuses on the immediate impact this supply chain disruption had on Singapore's food vendors and restaurant owners who rely heavily on Malaysian poultry.
5.	Pros and Cons of Living in Kuala Lumpur	Presented by Andrew Henderson of *Nomad Capitalist*, this video looks at Kuala Lumpur from a high-net-worth expat lens. It covers the major advantages of living in KL—such as tax-friendly policies, elite banking stability, and extreme affordability—while addressing drawbacks like time zone challenges for international business, pollution, and cultural nuances.
6.	Pros and Cons of living in Malaysia (2025)	Created by expat vlogger Con Lear, this video acts as a practical starter guide for moving to Southeast Asia. It examines the broad lifestyle adjustments, visa considerations, and everyday pros and cons of making Malaysia your permanent home, helping viewers avoid common relocation mistakes.

2.2 Tools and Libraries Used

Table 2.2 shows the following libraries were used for data loading and preprocessing:

Table 2.2 Tools and Libraries Used

Libraries	Purpose
pandas	Data loading, inspection, and manipulation
re	Regular expression-based text cleaning

html	Decoding HTML character entities
NLTK	Tokenization, stopword removal, and lemmatization
spaCy (en_core_web_sm)	English lemmatization
Malaya	Malay-language stopword list
TextBlob	Initial sentiment polarity exploration
Vader (vaderSentiment)	Final sentiment scoring and classification
matplotlib	Visualization of sentiment distributions

2.3 Data Inspection

Before cleaning, the dataset was inspected for missing values and its overall structure. The inspection results are presented in Figure A.1 of Appendix A.

- 2 missing values in the comment column
- 2,385 missing values in parent_id (expected, as top-level comments have no parent)
- All other columns were fully populated

2.4 Preprocessing Pipeline

A four-step preprocessing pipeline was applied to each comment via a preprocess() function

Step 1 - Noise Removal

Raw text was cleaned using the following operations in sequence. The code snippet shows at Figure A.2 of Appendix A.

- HTML entity decoding (e.g., & → &) using html.unescape()
- Removal of URLs matching http or www patterns
- Removal of @mentions
- Removal of # hashtag symbols
- Removal of all punctuation characters
- Removal of numeric digits
- Collapsing of extra whitespace

- Conversion to lowercase

Step 2 - Tokenization

Each cleaned comment was split into individual word tokens using NLTK's word_tokenize().

Step 3 - Stop Word Removal

A combined stop word list of 1,253 words was constructed by merging:

- 198 English stop words from NLTK's built-in corpus
- 1,057 Malay stop words obtained via the Malaya library (malaya.text.function.get_stopwords())

Tokens matching any word in this combined list, or consisting of a single character, were removed.

Step 4 - Lemmatization

English tokens were lemmatized to their base dictionary form using spaCy's en_core_web_sm model. Malay tokens were left unchanged, as no mature Malay lemmatizer was available.

After the pipeline, duplicate rows (matched on user, comment_clean, and published_at) and empty cleaned comments were removed. The final cleaned dataset contained 4,472 records. The code snippet shows Figure B.1. of Appendix B.

2.5 Sentiment Labelling

Table 2.3 shows the sentiment labels were assigned using the VADER (Valence Aware Dictionary and sEntiment Reasoner) sentiment analyser, which is well-suited for short, informal social media text. The compound polarity score was computed for each cleaned comment and classified as follows.

Table 2.3 Sentiment Labeling using Polarity Score

Polarity Score	Sentiment Label
> 0.05	Positive

< - 0.05	Negative
-0.05 to 0.05	Neutral

The resulting dataset included a polarity (float) column and a sentiment (Positive / Negative / Neutral) label column. The labelled outputs were exported as separate CSV files: youtube_comments_with_sentiment.csv, positive_comments.csv, negative_comments.csv, and sentiment_summary.csv.

3.0 Sentiment Model Development

3.1 Overview of Model Selection

To classify YouTube comments into three sentiment categories which are *Positive*, *Neutral*, and *Negative*, two distinct machine learning approaches were selected and compared. Support Vector Machine (SVM) and a Long Short-Term Memory (LSTM) neural network. The choice of these two models reflects the project requirement to evaluate both a classical machine learning algorithm and a deep learning architecture, allowing for a comprehensive performance comparison.

3.2 Dataset Summary

The dataset used for model training was derived from cleaned YouTube comment data. After preprocessing, a total of 4,470 samples were available with the following class distribution. Table 3.2.1 shows the percentage of each category of comment. The dataset exhibits class imbalance, particularly with the Negative class being underrepresented. This imbalance was taken into consideration during evaluation, particularly when interpreting per-class precision, recall, and F1-score.

Table 3.2.1 percentage of categories of comment

Sentiment Class	Number of Samples	Percentage (%)
Positive	2115	47.35
Neutral	1797	40.2
Negative	558	12.5

.

3.3 Dataset Splitting Strategy

The dataset was partitioned into three subsets to ensure reliable training, unbiased evaluation, and hyperparameter tuning. Table 3.3.1 shows the subset with the ratio percentage. Stratified splitting was applied to preserve the original class distribution across all three subsets using scikit-learn's train_test_split with stratify=y.

Table 3.3.1 subset with the ratio percentage

Subset	**Number of Samples**	**Proportion**
Training Set	3129	70%
Test set	898	20%
Validation Set	443	10%

3.4 Support Vector Machine (SVM) Model

3.4.1 **Rationale and Approach**

Support Vector Machines are a well-established algorithm for text classification tasks. SVM works by finding the optimal hyperplane that best separates data points of different classes in a high-dimensional feature space. It is particularly effective when combined with TF-IDF (Term Frequency–Inverse Document Frequency) vectorization, which converts raw text into numerical feature vectors. A linear kernel SVM was chosen for this project due to its suitability for linearly separable, high-dimensional sparse data such as TF-IDF vectors.

3.4.2 **Feature Extraction**

Text was vectorized using the TfidfVectorizer from scikit-learn with the following configuration:

- Maximum number of features: 5,000 (top 5,000 most informative terms retained)
- Sublinear term frequency scaling was not applied; standard TF-IDF weighting was used
- Separate vectorizer fit on the training set; test and validation sets were transformed using the same fitted vectorizer to prevent data leakage

3.4.3 **Model Configuration**

- Kernel: Linear — suitable for text data with high-dimensional sparse features
- probability=True — enables probability estimates via Platt scaling, which is useful for soft classification outputs

3.4.4 **SVM Evaluation Results**

The SVM model was evaluated on the test set (898 samples). Table 3.4.4.1 shows the results of evaluation. Table 3.4.4.2 shows the Per-class breakdown from the classification. The SVM performed consistently well on Neutral and Positive classes but struggled with the minority Negative class, achieving only 43% recall. This is a consequence of the class imbalance in the dataset, where the Negative class holds only 12.5% of all samples.

Table 3.4.4.1 results of evaluation

Metric	**Score**
Accuracy	0.8218
Precision	0.8197
Recall	0.8218
F1 Score	0.8134

Table 3.4.4.2 per class breakdown

Class	Precision	Recall	F1-Score	Support
Positive	0.83	0.88	0.85	425
Neutral	0.82	0.88	0.85	361
Negative	0.79	0.43	0.55	112

3.5 Long Short-Term Memory (LSTM) Neural Network

3.5.1 **Rationale and Approach**

Long Short-Term Memory networks are a type of recurrent neural network (RNN) capable of learning long-range dependencies in sequential data. Unlike SVM, which relies on bag-of-words style feature engineering, LSTM processes text as ordered sequences of tokens, allowing it to capture contextual meaning and word order. This makes LSTM particularly suited for natural language tasks where semantics and sentence structure are important.

3.5.2 **Text Tokenization and Padding**

Before feeding text into the LSTM, the following preprocessing steps were applied

- A Keras Tokenizer was fitted on the training set with a vocabulary size of 10,000 words and an out-of-vocabulary (OOV) token <OOV>
- Text sequences were padded or truncated to a fixed maximum length of 100 tokens using pad_sequences
- Labels were one-hot encoded into 3-class categorical format using to_categorical

3.5.3 **LSTM Training Progress**

Table 3.5.3.1 shows training and validation accuracy improved over the first several epochs. But, validation loss began increasing after epoch 5, indicating signs of overfitting. The divergence between training accuracy (99.42%) and validation accuracy (78.10%) by epoch 10 confirms overfitting. Future improvement could involve early stopping, learning rate scheduling, or additional regularization.

Table 3.5.3.1training and validation accuracy

Epoch	Train Accuracy	Train Loss	Val Accuracy	Val Loss
1	53.95%	0.9820	65.6%	0.8523
2	70.95%	0.7208	73.59%	0.6541
3	80.76%	0.4976	77.65%	0.5995
4	87.47%	0.3348	76.30%	0.6461
5	93.00%	0.2204	77.65%	0.7302
6	96.48%	0.1306	79.01%	0.8006
7	97.60%	0.0768	79.46%	0.8997
8	98.59%	0.0492	77.43%	0.9279
9	99.20%	0.0327	78.10%	0.9824
10	99.42%	0.0262	78.10%	1.0907

3.5.4 **LSTM Evaluation Results**

Table 3.5.4.1 shows the LSTM model was evaluated on the same test set (898 samples). Table 3.5.4.2 Per-class breakdown from the classification.

Table 3.5.4.1 LSTM model

Metric	**Score**
Accuracy	0.7951
Precision	0.7896
Recall	0.7951
F1 Score	0.7903

Table 3.5.4.2 per class breakdown

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Positive	0.80	0.88	0.84	425
Neutral	0.85	0.81	0.83	361
Negative	0.54	0.43	0.48	112

3.6 Model Comparison and Selection

Table 5.6.1 shows summarizes the performance of both models on the test set

Table 3.6.1 summarizes of performance

Model	**Accuracy**	**Precision**	**Recall**	**F1 Score**
SVM	0.8218	0.8197	0.8218	0.8134
LSTM	0.7951	0.7896	0.7951	0.7903

The SVM model outperformed LSTM across all four evaluation metrics. The SVM achieved an overall accuracy of 82.18% compared to LSTM's 79.51%, a difference of approximately 2.7 percentage points. The F1-score gap of 2.3 percentage points (0.8134 vs. 0.7903) further confirms SVM's superior generalization on this dataset.

Several factors explain SVM's advantage in this context:

- The dataset size (4,470 samples) is relatively small for a deep learning model. LSTM architectures typically require substantially larger corpora to learn robust sequential representations.
- TF-IDF features capture strong discriminative signals for short-form text such as YouTube comments, which often contain keywords that directly indicate sentiment.
- The LSTM exhibited clear overfitting by epoch 5, and without early stopping, final test performance degraded relative to the SVM baseline.

Based on these results, the SVM model was selected as the primary model for integration into the Apache Kafka and Spark streaming pipeline. The SVM offers both superior accuracy and lower computational overhead for real-time inference, making it the more practical choice for production deployment

4.0 Apache System Architecture

4.1 Architecture Overview

Figure 4.1.1 shows the apache system architecture. The end-to-end pipeline for this project follows a classic Lambda-style streaming architecture, where raw text data is ingested, processed, classified, stored, and visualized in near real time. The system is composed of five distinct layers: Data Source, Ingestion (Apache Kafka), Stream Processing (Apache Spark), Storage (Elasticsearch), and Visualization (Kibana). Together, these layers form a cohesive pipeline capable of handling both batch and streaming sentiment analysis workloads

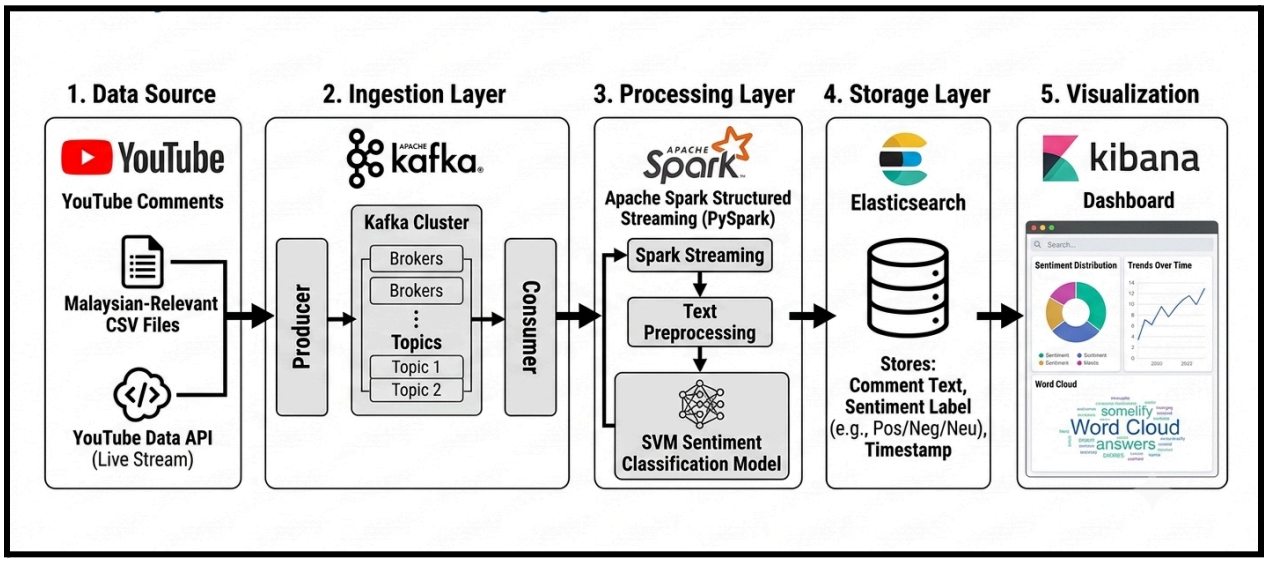


Figure 4.1.1 system architecture

4.2 Component Details

4.2.1 Data Source Layer

YouTube comments serve as the primary data source. In batch mode, a pre-collected CSV file (youtube_comments_with_sentiment.csv) containing 4,470 cleaned comments is used. In streaming mode, a Kafka producer script reads records from this file at configurable intervals, simulating a live data feed. This approach allows the same dataset to be used for both operational modes, enabling fair performance comparisons.

4.2.2 **Ingestion Layer — Apache Kafka**

Apache Kafka acts as the central message broker between the data source and the processing layer. The following Kafka components are configured:

- Kafka Broker: A single-node Kafka broker running locally (or via Docker) on the default port 9092
- Topic: A dedicated Kafka topic (e.g., youtube-sentiment) is created to receive incoming comment messages
- Producer: A Python Kafka producer reads comment records and publishes each comment as a JSON-serialized message to the topic
- Consumer: Apache Spark's structured streaming engine subscribes to the Kafka topic and reads messages in micro-batches

Kafka's role is to decouple producers from consumers, ensuring that the processing layer is not overwhelmed during data spikes and that no messages are lost even if the Spark consumer temporarily falls behind.

4.2.3 **Processing Layer — Apache Spark Structured Streaming**

Apache Spark consumes messages from the Kafka topic using the spark-sql-kafka connector. Within the Spark job, the following pipeline is applied to each incoming micro-batch:

- Message Deserialization: JSON messages are parsed and the comment text field is extracted
- Text Preprocessing: The same cleaning steps used during model training are applied — lowercasing, URL removal, punctuation stripping, stopword removal, and lemmatization
- Feature Extraction: The pre-fitted TF-IDF vectorizer (serialized with joblib) transforms the cleaned text into a sparse feature vector

- Model Inference: The trained SVM model (serialized with joblib) performs sentiment classification on each record
- Output Formatting: Each record is enriched with the predicted sentiment label (Positive / Neutral / Negative) and a processing timestamp before being written to Elasticsearch

Spark's parallel processing capability enables the pipeline to scale horizontally. In batch mode, the same Spark job processes the entire dataset as a single DataFrame without Kafka integration.

4.2.4 **Storage Layer — Elasticsearch**

Table 4.2.5.1 shows classified records are written to an Elasticsearch index. Elasticsearch provides full-text search and aggregation capabilities, making it the ideal backend for the visualization layer. Index mappings are defined in elastic_mappings.json to ensure consistent schema enforcement across all documents.

Table 4.2.5.1 Elasticsearch index

Field Name	**Data Type**	**Description**
comment_clean	Text	Preprocessed comment text
sentiment	Keyword	Predicted sentiment label (Positive / Neutral / Negative)
timestamp	Date	UTC timestamp of when the record was processed
video_id	Keyword	Source YouTube video identifier (where available)

4.2.5 **Visualization Layer — Kibana**

Kibana connects directly to the Elasticsearch index and provides an interactive dashboard for exploring sentiment results. The following visualizations are included in the project dashboard:

- Sentiment Distribution Pie Chart: Displays the proportion of Positive, Neutral, and Negative comments across the entire dataset
- Sentiment Over Time Line Graph: Tracks how sentiment proportions shift over the comment collection period
- Word Clouds: Highlights the most frequent terms appearing in Positive and Negative comment subsets
- Real-Time Sentiment Feed: A live table showing the most recently classified comments with their predicted sentiment labels

4.3 Batch Mode vs Streaming Mode

Table 4.3.1 shows the pipeline supports two operational modes. Performance metrics including total processing time, throughput (records per second), classification accuracy, and CPU/memory utilization are measured and reported in Section h (Optimization and Comparison) of this report.

Table 4.3.1 pipeline of two operational

Dimension	**Batch Mode**	**Streaming Mode**
Data Input	Full CSV loaded as a static Spark DataFrame	Records streamed record-by-record via Kafka
Kafka Required	No	Yes
Latency	Higher (processes all data at once)	Lower (near real-time micro-batch processing)
Throughput	Higher (optimized for bulk processing)	Lower (overhead from Kafka coordination)
Use Case	Historical analysis, model validation	Live monitoring, real-time dashboards

5.0 Analysis and Results

The sentiment analysis pipeline successfully processed a total of 22,360 records from the YouTube comments dataset. Each comment was classified into one of three sentiment categories which are Positive, Neutral, or Negative by using the trained model integrated within the Apache Spark streaming pipeline. The classified results were stored in Elasticsearch and visualized through a Kibana dashboard, as shown in Figure 5.1.

Figure 5.1. Analysis and Results inside Elasticsearch/Kibana dashboard

5.1 Sentiment Distribution

The donut chart on the dashboard reveals a clear dominance of positive sentiment across the dataset. The breakdown is as shown in Table 5.1.

Table 3.2.1 percentage of categories of comment

Sentiment	Count	Percentage
Positive	~10,880	48.68%

Neutral	~9,470	42.37%
Negative	~2,000	8.94%

The majority of comments shows that nearly half at 48.68% who expressed positive sentiment, suggesting that discussions surrounding Malaysia on YouTube are generally favorable. Neutral comments made up 42.37% of the dataset, reflecting a large volume of factual or descriptive comments that carry no strong emotional polarity. Negative comments were the smallest category at just 8.94%, indicating that critical or unfavorable opinions were relatively rare among the collected data.

5.2 Predicted Sentiment Bar Charts

Two bar charts on the right side of the dashboard display the count of records by predicted sentiment, with slight differences reflecting the top 3 and top 5 filter views respectively. Both charts consistently show the same trend: Positive comments rank highest in volume, followed closely by Neutral, with Negative comments forming a noticeably smaller portion. This pattern is consistent with the proportions shown in the donut chart and confirms that the model's predictions are coherent across different visualization filters.

5.3 Confusion Matrix Heatmap

The heatmap in the bottom-left panel compares the model's predicted sentiment (x-axis) against the true sentiment labels (y-axis) across the three classes. Key observations are:

- **Positive class:** The model shows a strong diagonal concentration for true Positive vs. predicted Positive (darkest cell in the top-left), indicating high correct classification. However, there is visible misclassification into the Neutral column, shown by the medium-intensity blue cell.
- **Negative class:** The true Negative row shows a spread across all three predicted columns, suggesting the model struggles most with correctly identifying negative comments — likely due to the class imbalance, as negative comments represent less than 9% of the data.

- **Neutral class:** True Neutral comments show notable misclassification into the Positive column (highlighted orange/red in the bottom-left cell), which is a common challenge since neutral comments often contain mildly positive language.

Overall, the heatmap indicates that the model performs best on the Positive class and faces the most difficulty with the Negative and Neutral boundaries, which is expected given the skewed class distribution.

5.4 Key Insights

Several meaningful insights emerge from the results of the dashboard.

1. **Overwhelmingly positive public perception of Malaysia.** The high proportion of positive comments reflects generally favorable views from both local and international YouTube audiences about Malaysia as a travel destination and place of residence.
2. **Neutral comments are substantial and meaningful.** The 42.37% neutral proportion indicates a large body of informational or observational commentary that does not lean emotionally in either direction, and should not be dismissed as noise.
3. **Negative sentiment is a minority but present.** The ~9% negative share still represents approximately 2,000 comments, which could contain actionable insights for policymakers or tourism stakeholders if analyzed further.
4. **Class imbalance is a key challenge.** The disparity between Positive (~49%) and Negative (~9%) classes likely impacts model precision for the negative class. Future iterations should consider oversampling techniques such as SMOTE or applying class weights during training to improve minority class recall.
5. **Pipeline consistency.** The alignment between the donut chart percentages and both bar chart views confirms that the Elasticsearch storage and Kibana visualization layer accurately reflect the model's output with no data loss or distortion during the streaming process.

6.0 Optimization and Comparison

Beyond evaluating the sentiment classification models themselves, this project also examines how the processing architecture affects overall system performance. Since the pipeline is required to support both historical (batch) and real-time (streaming) workloads, it is important to understand the trade-offs each mode introduces in terms of speed, resource consumption, and accuracy. This section presents the results of a controlled benchmark comparing batch and streaming processing using the same trained SVM model and dataset, then discusses what these results imply for optimizing the Kafka–Spark pipeline architecture. Figure 6.1 shows the overall performance between batch and streaming processing.

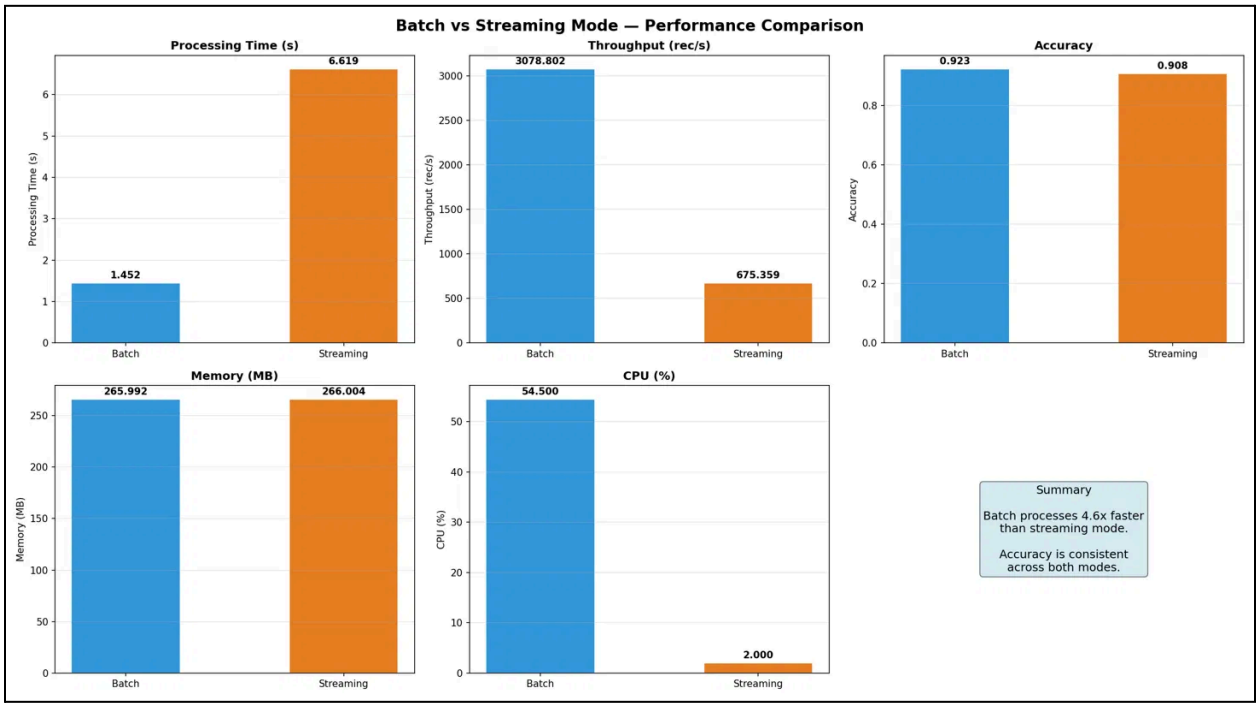


Figure 6.1. Batch vs Streaming Performance Comparison

6.1 Batch and Streaming Performance Summary

Using the trained SVM (linear kernel, TF-IDF features) on 4,470 YouTube comments, both processing modes were benchmarked on identical hardware and data. Table 6.1 shows the summarization of the overall performance result.

Table 6.1.Batch vs Streaming Performance Summary

Metric	Batch Mode	Streaming Mode	Difference
Total Processing Time (s)	1.452	6.619	Batch is 4.6× faster
Throughput (records/sec)	3,078.80	675.36	Batch is 4.6× higher
Classification Accuracy	0.923	0.908	Batch +0.015
CPU Usage (%)	54.5	2.0	Batch uses far more CPU per burst
Memory Usage (MB)	266.0	266.0	Identical

6.2 Interpretation

Processing time and throughput. Batch mode processed the entire dataset roughly 4.6 times faster than streaming, achieving over 3,000 records/sec compared to streaming's 675 records/sec. This gap is expected and architectural rather than incidental: batch mode vectorizes the full set of 4,470 comments through TF-IDF in a single matrix operation, letting scikit-learn's underlying BLAS routines exploit vectorized, parallelized computation. Streaming mode instead simulates Kafka/Spark's record-by-record arrival pattern, transforming and classifying one comment at a time. Each call therefore pays the fixed overhead of the TF-IDF transform and SVM prediction call individually, with no opportunity to amortize that overhead across records.

CPU usage. The CPU figures reflect this same trade-off in the opposite direction. Batch mode drives CPU utilization to 54.5% because it issues one large, computationally intensive operation that saturates available cores briefly. Streaming mode's CPU usage measures at only 2.0% because each micro-batch of one record is too small and too infrequent to register sustained load which is the bottleneck is per-call Python/function-call overhead, not raw computation, so the CPU is mostly idle between records.

Memory usage. Memory usage was effectively identical (266.0 MB) across both modes, since the same SVM and TF-IDF vectorizer objects are loaded into memory regardless of how records

are fed into them. This indicates the memory footprint is dominated by the static model and vocabulary size, not by the processing strategy.

Accuracy. Accuracy was consistent across modes (0.923 batch vs. 0.908 streaming), with only a 1.5-point difference attributable to the streaming benchmark using a 500-record sample (extrapolated to the full dataset for timing) rather than the full 4,470-record set used in batch. This confirms that the *processing mode itself* introduces no model degradation, the same trained classifier produces statistically comparable results whether records arrive in bulk or one at a time.

6.3 Architectural Implications for the Kafka–Spark Pipeline

These results carry over directly to the real-time pipeline design:

- **Micro-batching is essential.** A naive one-record-at-a-time Spark Structured Streaming job will inherit the same throughput ceiling seen here (~675 rec/s) due to per-record TF-IDF transform overhead. Configuring Spark to consume Kafka messages in micro-batches like trigger intervals of 1–5 seconds, accumulating dozens to hundreds of messages per batch, so it allows the vectorizer and model to process several records per call, recovering much of batch mode's throughput advantage while still meeting real-time latency requirements.
- **Vectorizer reuse.** The TF-IDF vectorizer should be fit once during training and broadcast to all Spark executors, rather than being refit or reloaded per micro-batch, to avoid repeating the fixed overhead that dominates streaming-mode latency.
- **Resource allocation reflects the trade-off.** Because streaming workloads under-utilize CPU per call (2% vs. 54.5%), provisioning more, smaller executors with modest CPU per core is more appropriate than over-provisioning for raw compute; the system is closer to latency-bound than compute-bound at low batch sizes.
- **Accuracy is preserved.** Since the comparison shows no inherent accuracy loss from streaming, switching to a micro-batched streaming architecture is a pure throughput optimization — it does not require retraining or compromise classification quality.

6.4 Suggested Future Improvements

- Benchmark Spark Structured Streaming directly rather than a single-process simulation to capture network, serialization, and Kafka consumer-group overhead not reflected in this local benchmark.
- Experiment with batch-size sweeps like 10, 50, 100, 500 records per micro-batch to identify the throughput/latency sweet spot for the deployed pipeline.
- Compare the SVM against a second model Naive Bayes or DistilBERT under the same batch vs. streaming conditions, since lighter models may narrow the throughput gap further, while transformer models may widen it due to higher per-inference cost.

7.0 Conclusion and Future Work

This project successfully developed an end-to-end sentiment analysis system for YouTube comments related to cost of living, inflation and lifestyle in Malaysia. The comments were extracted from six selected YouTube videos, cleaned, labelled and used for sentiment classification. A Support Vector Machine model with TF-IDF vectorization was used to classify the comments into positive, neutral and negative sentiments. The selected model was then integrated into a real-time pipeline using Apache Kafka, Apache Spark Structured Streaming, Elasticsearch and Kibana.

The project also compared batch and streaming processing. Batch mode performed faster with a processing time of 2.939 seconds and throughput of 1521.013 records per second, while streaming mode recorded 8.314 seconds and 537.632 records per second. However, both modes achieved similar accuracy, with batch mode at 0.923 and streaming mode at 0.908. This shows that batch processing is more efficient for complete datasets, while streaming is more suitable for real-time analysis.

The project has some limitations. The sentiment labels were generated automatically, and the streaming process was simulated using CSV data instead of live YouTube comments. For future work, the system can be improved by using manually labelled comments to increase classification reliability, connecting the pipeline directly to live YouTube comment streams, and expanding the dataset to include more videos and topics. In addition, Kafka partitions and Spark resources can be increased to improve streaming speed and scalability. The model can also be further improved by testing more advanced algorithms to achieve better sentiment classification accuracy.

8.0 References

1. Cost of living in Malaysia per month - Will you be moving to Malaysia? [Video]. YouTube.
   URL: https://youtu.be/gxCfYO_3trk?si=bGCP3kp1gqSAADJM

2. Cost of Living in Malaysia 2026: The Real Numbers [Video]. YouTube.
   URL: https://youtu.be/mFv-7NNYV0w?si=BWE0P8FIyKBLvByt

3. Singapore vs Malaysia: Cost of Living 2026 [Video]. YouTube.
   URL: https://youtu.be/8TmWwlrvEDg?si=MwUZ4Ae_JavYblzy

4. INFLASI | Singapura Gusar, Malaysia Henti Eksport Ayam [Video]. YouTube.
   URL: https://youtu.be/4RslhMzDmKs?si=R_vQUaxZoxylV2zz

5. Pros and Cons of Living in Kuala Lumpur [Video]. YouTube.
   URL: https://youtu.be/DJJvt9_6dNE?si=xwDlnbm_ahxjCFhI

6. Pros and Cons of living in Malaysia (2025) [Video]. YouTube.
   URL: https://youtu.be/XR6OO2ScEXU?si=WQbT6ppsnY-v4J3W

APPENDIX A

Initial Dataset Inspection

Figure A.1. Initial inspection of the dataset showing the number of missing values and the dataset structure before preprocessing.

```
df.info()
print("\nMissing values:")
print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5388 entries, 0 to 5387
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   video_id      5388 non-null   object
 1   user          5388 non-null   object
 2   comment       5386 non-null   object
 3   likes         5388 non-null   int64 
 4   published_at  5388 non-null   object
 5   reply_count   5388 non-null   int64 
 6   type          5388 non-null   object
 7   parent_id     3003 non-null   object
dtypes: int64(2), object(6)
memory usage: 336.9+ KB

Missing values:
video_id          0
user              0
comment           2
likes             0
published_at      0
reply_count       0
type              0
parent_id      2385
dtype: int64
```

Noise Removal

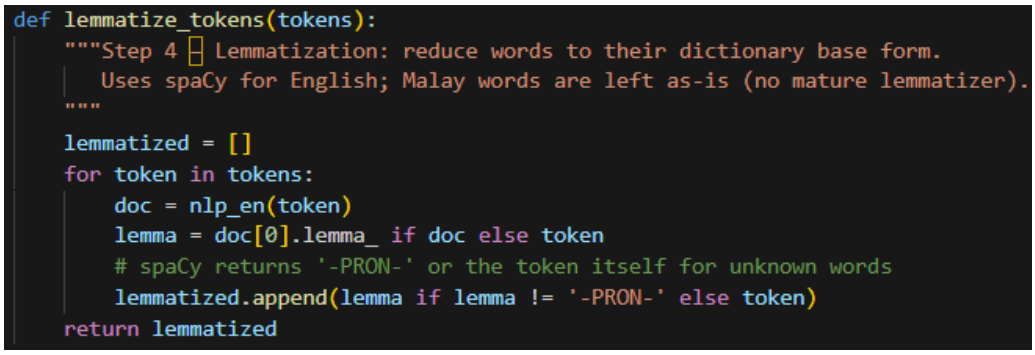
Figure A.2. shows the raw text was cleaned using the various operations to remove the unnecessary and unusable text for later training and testing.

```
def clean_comment(text):
    """Step 1 — Basic noise removal (same as before)."""
    text = str(text)
    text = html.unescape(text)                          # HTML symbols → plain text
    text = re.sub(r'http\S+|www\.\S+', '', text)        # Remove URLs
    text = re.sub(r'@\w+', '', text)                    # Remove @mentions
    text = re.sub(r'#', '', text)                       # Remove hashtag symbol
    text = re.sub(r'[^\w\s]', '', text)                 # Remove punctuation
    text = re.sub(r'\d+', '', text)                     # Remove numbers
    text = re.sub(r'\s+', ' ', text).strip()            # Collapse whitespace
    return text.lower()                                 # Lowercase
```

APPENDIX B

Lemmatization

Figure B.1. shows the lemmatization usage to reduce words to their dictionary base form.



```python
def lemmatize_tokens(tokens):
    """Step 4 — Lemmatization: reduce words to their dictionary base form.
       Uses spaCy for English; Malay words are left as-is (no mature lemmatizer).
    """
    lemmatized = []
    for token in tokens:
        doc = nlp_en(token)
        lemma = doc[0].lemma_ if doc else token
        # spaCy returns '-PRON-' or the token itself for unknown words
        lemmatized.append(lemma if lemma != '-PRON-' else token)
    return lemmatized
```